

# USER AUTHENTICATION DEVELOPMENT

Created by Humais Ansari (Backend)

## INTRODUCTION

This document outlines the procedural steps undertaken to construct a robust user authentication system with a focus on security. Additionally, it highlights the testing procedures employed and indicates the backend teams next steps.

## DEVELOPMENT PROCESS

### Backend Application Technologies

- Java21
- Spring boot & Spring security 6 (java framework)

### High level overview of code

I structured the project into distinct packages and classes following established conventions. Below is a concise overview of each class. For comprehensive code details, please refer to the backend branch on Git, located within the backend - authenticationBackend directory.

#### AuthenticatedBackendApplication (class)

- The AuthenticatedbackendApplication class is the entry point for the spring boot application, responsible for bootstrapping the application context and initialising essential roles and an admin user in the database during startup. It demonstrates the use of Spring Boot's bean creation mechanism and integration with Spring Data repositories and security-related functionality.

#### SecurityConfiguration (class within **configuration** package)

- This class orchestrates the configuration of security features such as password encoding, authentication manager, JWT token handling, and HTTP security rules for the Spring Boot application. It ensures that the application's endpoints are protected, and access is granted based on authentication and authorisation rules.

#### AdminController (class within **controllers** package)

- This class defines a controller specifically for handling requests related to administrative tasks. It provides an endpoint (**/admin/**) that returns a message indicating admin-level access, and it can be further expanded to include more complex logic and functionality related to administrative operations.

AuthenticationController (class within **controllers** package)

- This class defines a controller responsible for handling user registration and login requests. It delegates the actual authentication logic to the **AuthenticationService** and returns appropriate responses based on the outcome of the authentication process.

UserController (class within **controllers** package)

- This class defines a controller responsible for handling requests related to user-specific functionalities. In this case, it provides an endpoint to demonstrate user-level access.

ApplicationUser (class within **models** package)

- This class encapsulates the user-related data and behaviour, making it suitable for use with Spring Security's authentication and authorization mechanisms. It represents a user entity in the application's persistence layer.

LoginResponseDTO (class within **models** package)

- The LoginResponseDTO class facilitates the transmission of authentication-related data from the backend to the client/frontend application after a successful login attempt. It helps maintain a clean separation of concerns by encapsulating related data into a single object.

RegistrationDTO (class within **models** package)

- The RegistrationDTO class serves as a data transfer object (DTO) used for carrying registration information from the frontend/client to the backend during user registration processes.

Roles (class within **models** package)

- This class represents user roles within the application and implements the GrantedAuthority interface provided by Spring Security.

RoleRepository (interface within **repository** package)

- The RoleRepository interface simplifies the interaction with the database for managing roles within the application, offering a higher level of abstraction and productivity compared to manual SQL queries or DAO implementations.

UserRepository (interface within **repository** package)

- The UserRepository interface simplifies the interaction with the database for managing users within the application, offering a higher level of abstraction and productivity compared to manual SQL queries or DAO implementations.

AuthenticationService (class within **services** package)

- This class is responsible for user authentication-related operations, such as user registration and login.

TokenService (class within **services** package)

- The TokenService class is responsible for generating JWT (JSON Web Token) tokens, which can be used to authenticate and authorize users within the application.

UserService (class within **services** package)

- The UserService class is responsible for loading user details during the authentication process, thereby enabling secure and reliable user authentication within the application.

KeyGeneratorUtility (class within **utils** package)

- This class provides a convenient and reusable mechanism for generating RSA key pairs, facilitating secure encryption and authentication mechanisms within the application. Its encapsulation of key generation logic promotes code maintainability and ensures consistent generation of cryptographic keys.

RsaKeyProperties (class within **utils** package)

- This class serves as a central repository for RSA key properties within the application, facilitating key generation, storage, and retrieval. Its Spring component nature allows for easy integration with other components that require access to RSA keys for cryptographic operations.

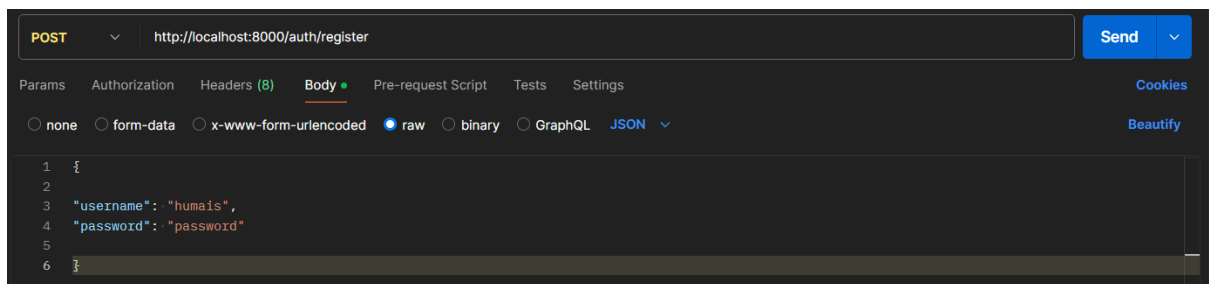
Application.properties (Resources directory)

- This configuration sets up a Spring Boot application named "authenticatedbackend" with an embedded server listening on port 8000. It connects to a MySQL database hosted on "aws.connect.psdb.cloud" (planet scale), and it configures Hibernate to manage the database schema automatically

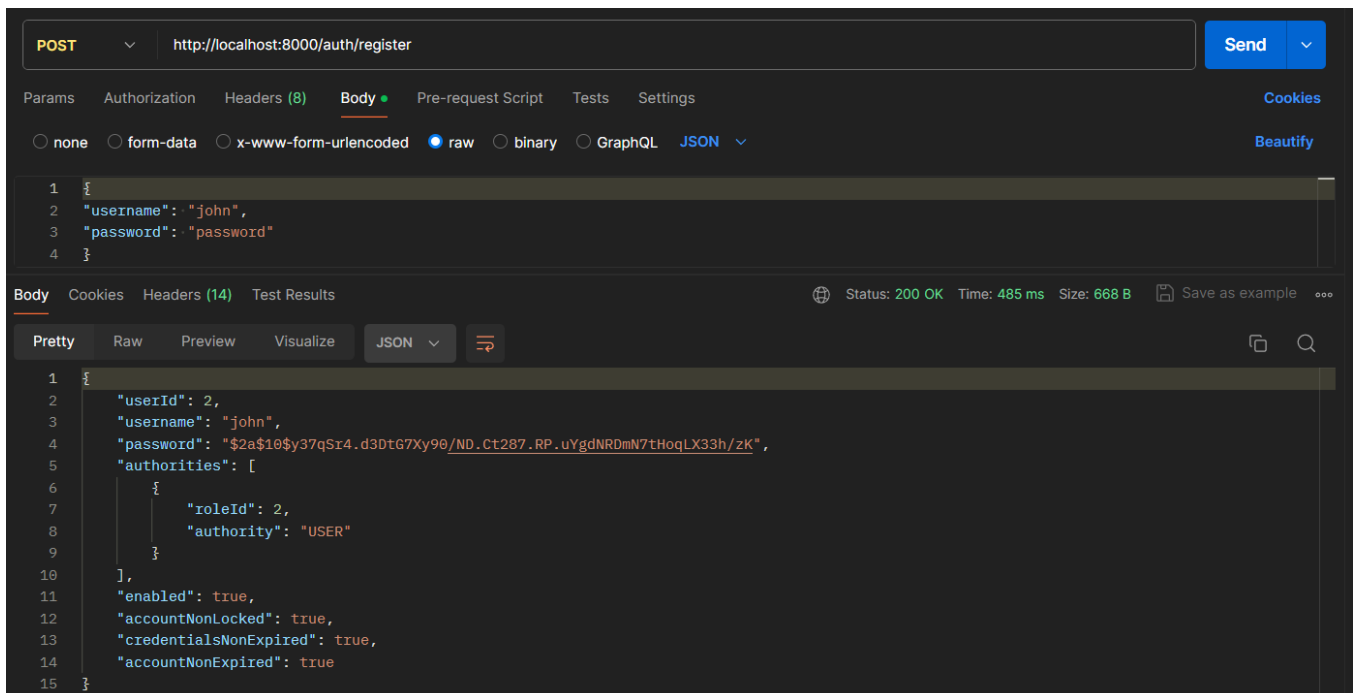
## TESTING

I utilised Postman, an API client tool renowned for testing, debugging, and documenting APIs, to validate the functionality of the user authentication backend application. Below are screenshots captured from the Postman client, illustrating a detailed walkthrough of how a hypothetical client would engage with the backend system.

### User registration



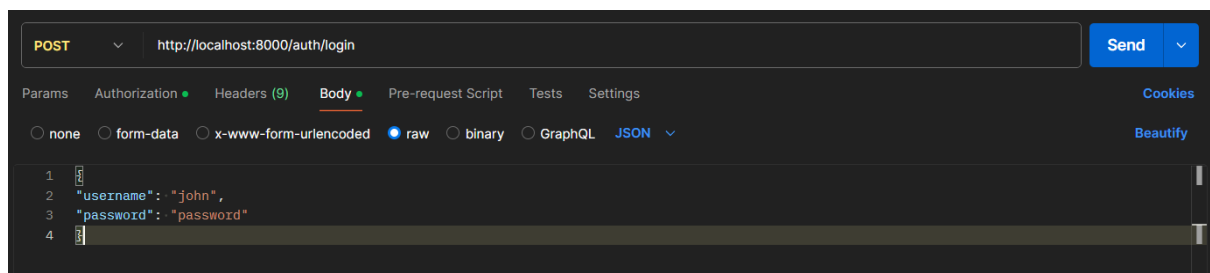
In the registration process, users will navigate to the 'auth/register' route where they will input their chosen username and password. For example, the username might be 'john' and the password 'password', both provided in JSON format. Upon submission, the user's registration will be processed and completed.



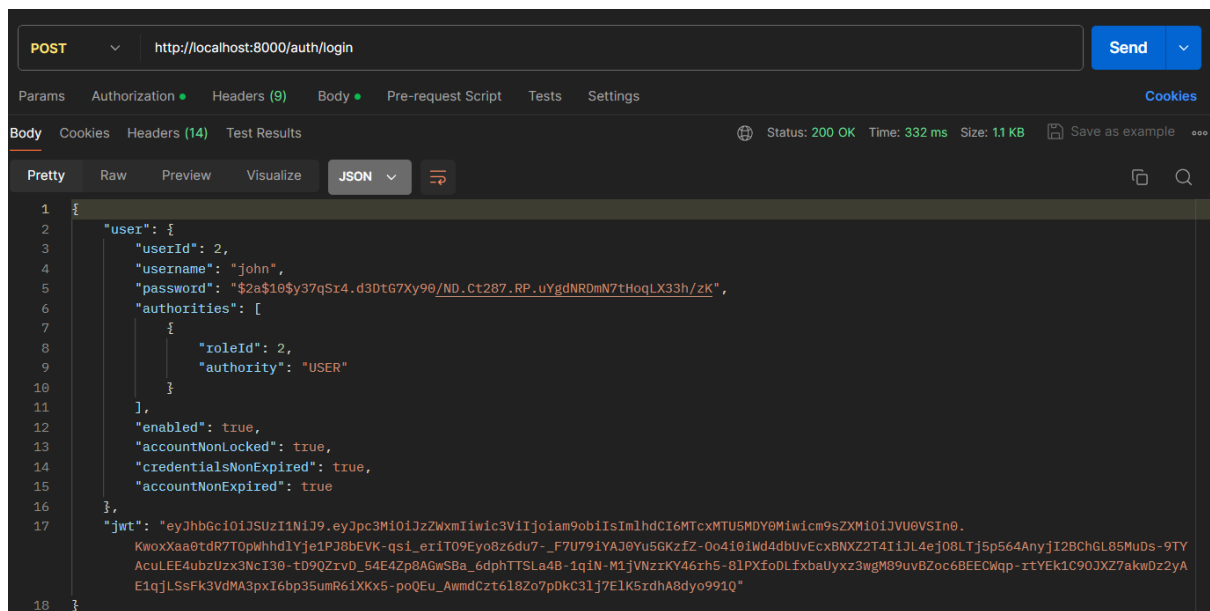
This returns a status of 200, which highlights the connection has gone through. We can now view various properties related to this users account, including: userId, username, password, roles, authority, and other account details. Notably, the password that will be stored into the database is hashed, which was specifically instructed and recommended by our project supervisor James.

Furthermore, this account has an authority of “USER” which highlights that it can only access user level routes. I will explore and demonstrate this functionality in the ‘user login’ section.

## User Login



In the login process, users will be directed to the 'auth/login' route, where they will input their username and password. For instance, the username might be 'john' and the password 'password', provided in JSON format. Upon submission, the user will be authenticated and logged in.

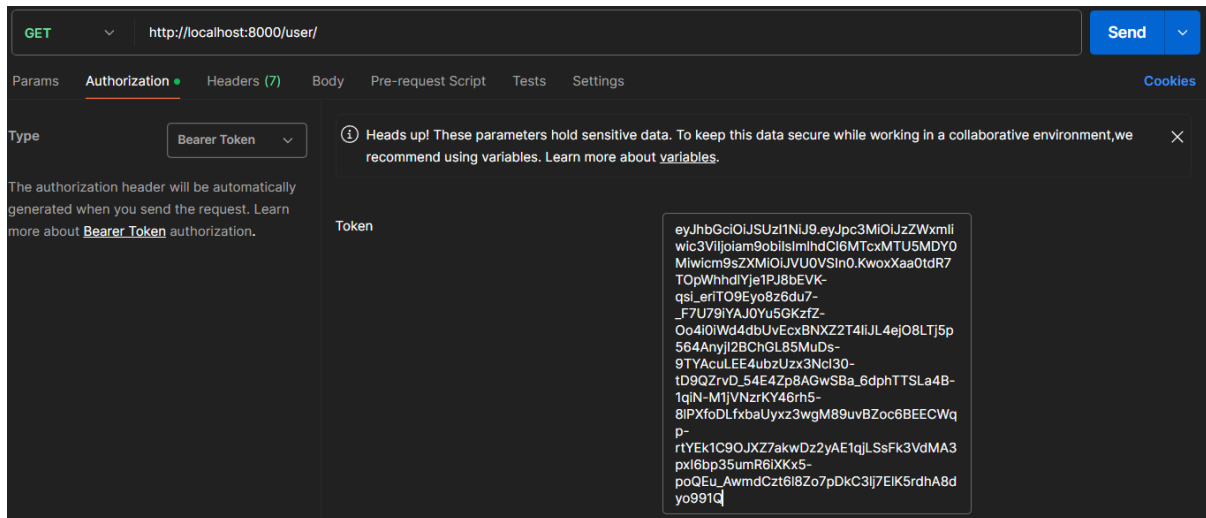


This returns a status of 200, which highlights the connection has gone through.

Similarly to 'User registration', this displays various properties related to the user's account.

Additionally, JWT provided in the JSON data is a token issued by a server (issuer) to represent the user "john" with certain roles and permissions (claims).

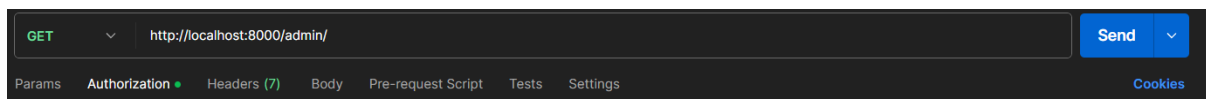
I will now show what access level this user has.



I've incorporated the token received upon logging in into the 'user/' route to showcase its functionality. Upon clicking 'send', the response indicates that this account has 'user access level privileges'.



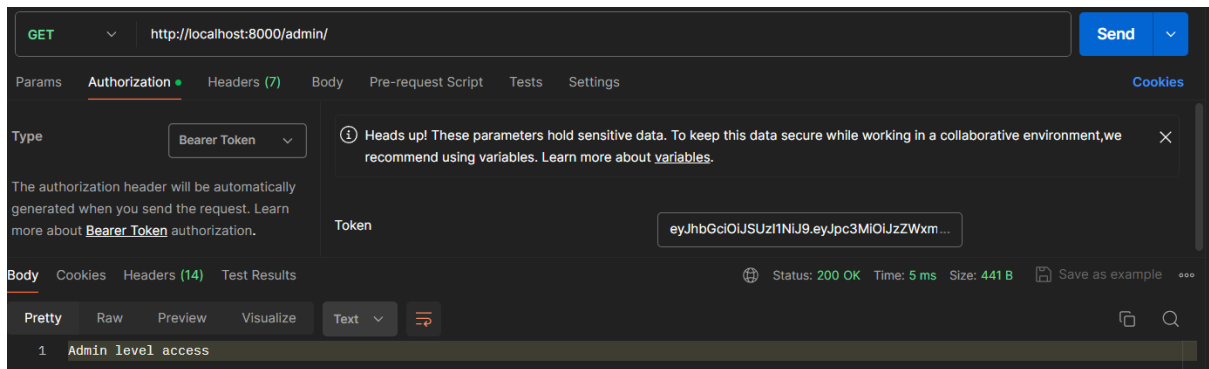
I will now repeat this but this time with the 'admin/' route.



When clicking send, it returns a 403 error as this account only has access to user level, and not access to admin level.



I will now show this process with a jwt generated for an admin.



This JWT signifies that this admin account has 'admin level access', as only administrators have authorisation to access this route, unlike regular users.

In conclusion, the testing conducted using the Postman client for the Spring Boot user-authenticated backend application has been completed. Overall, the test was successful, meeting the authentication requirements set for the application.

## FRONTEND TESTING

In this section, I'll showcase how the login form on the frontend interacts with the user authentication backend application.

```
const handleSubmit = async (event) => {
  event.preventDefault(); // Prevents the default form submission behavior

  try {
    const response = await fetch('http://localhost:8000/auth/login', {
      method: 'POST', // Sends a POST request to the specified URL
      headers: {
        'Content-Type': 'application/json', // Specifies that the request body is JSON
      },
      body: JSON.stringify({ username, password }), // Converts username and password to JSON string and sets it as the request body
    });

    if (!response.ok) { // Checks if the response status is not within the 200-299 range
      const errorData = await response.json(); // Parses the response body as JSON
      throw new Error(errorData.message || 'Authentication failed.');// Throws an error with the message from the response or a default message
    }

    const responseData = await response.json(); // Parses the response body as JSON
    const { jwt } = responseData; // Destructures the 'jwt' property from the response data

    if (!jwt) { // Checks if the JWT token is missing
      throw new Error('Authentication token not found.');// Throws an error if the JWT token is missing
    }

    localStorage.setItem('token', jwt); // Stores the JWT token in the browser's local storage
    navigate('/dashboard'); // Navigates to the '/dashboard' route
  } catch (error) {
    setOpenSnackbar(true); // Sets the state to open the snackbar
    setErrorMessage(error.message); // Sets the error message to be displayed in the snackbar
  }
};
```

This is the relevant logic that has been implemented within the 'Login Component' inside the react frontend application.

## Welcome Back!

Please enter your details below to log in

Username \*  
admin

Password \*  
password

LOGIN

[Sign Up](#)

[Forgot Password](#)

In this example, we will be logging in with an admin level account of username 'admin' and password 'password'. A successful login allows you to navigate to the dashboard route.

When clicking submit the user successfully connects to the dashboard. The username and **hashed password** of the account can then be viewed within an actual database, by running a 'select \* from users;' query.

```
4 • Select * from users;
```

| Result Grid                                                                                              |         |                                                |          |
|----------------------------------------------------------------------------------------------------------|---------|------------------------------------------------|----------|
| Filter Rows: <input type="text"/>                                                                        |         |                                                |          |
| Export:               |         |                                                |          |
| Wrap Cell Content:  |         |                                                |          |
|                                                                                                          | user_id | password                                       | username |
| ▶                                                                                                        | 1       | \$2a\$10\$RYzPLIMaAenxTsLzXDtyerBsvNjrVo6iB... | admin    |
|                                                                                                          | 2       | \$2a\$10\$y37qSr4.d3DtG7Xy90/ND.Ct287.RP.uY... | john     |

Furthermore, we can also view the roles (authority) of the users in our database by running the query below.

```
14 • SELECT u.user_id, u.username, r.authority
15 FROM users u
16 JOIN user_role_junction urj ON u.user_id = urj.user_id
17 JOIN roles r ON urj.role_id = r.role_id;
18
```

| Result Grid                                                                                              |         |          |           |
|----------------------------------------------------------------------------------------------------------|---------|----------|-----------|
| Filter Rows: <input type="text"/>                                                                        |         |          |           |
| Export:              |         |          |           |
| Wrap Cell Content:  |         |          |           |
|                                                                                                          | user_id | username | authority |
| ▶                                                                                                        | 1       | admin    | ADMIN     |
|                                                                                                          | 2       | john     | USER      |

To showcase a scenario where the user attempts to login with an incorrect username or password, an “authentication token not found” error is thrown. (This can be changed to better align with what a user would likely understand like “incorrect username or password”).

## Welcome Back!

Please enter your details below to log in

Username \*

Spongebob

Password \*

patrick

LOGIN

[Sign Up](#)[Forgot Password](#)

Authentication token not found. X

© 2024 Ecomute -

This concludes the testing section where I’ve highlighted how the frontend interacts with the backend, showing successful user authentication.

## CONCLUSION & NEXT STEPS

To conclude, this document outlines a successful user authentication system designed to interact between a front-end application and the backend.

In addition to implementing logic on the frontend login screen to establish connections with the backend, further frontend testing will be conducted to assess the functionality of the sign-up screen. This entails the implementation of backend connectivity logic, a task assigned to the backend team (Humais).

Moreover, collaborative discussions will be held with other sub teams to determine the preferred method of user login, whether through email or username. Additionally, the backend team will prioritise enhancing the signup process to accommodate the storage of additional user details, such as emails, in the database upon successful registration.

Overall, we look iteratively develop the user authentication system when required.

*Developed, tested, and documented by Humais Ansari (Team39 Backend)*